



Faculdade de Ciências da Universidade de Lisboa

# Break-it Phase Vulnerability Analysis

## Secure Gallery Log Systems

José Sampaio, fc66191  
Rodrigo Neto, fc59850  
Gabriel Cambraia, 58671

## Contents

|          |                                                                                      |          |
|----------|--------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                                  | <b>3</b> |
| <b>2</b> | <b>Evaluation Criteria Used</b>                                                      | <b>3</b> |
| <b>3</b> | <b>Methodology: Behavioral and Security Testing</b>                                  | <b>3</b> |
| <b>4</b> | <b>Implementation G21: Group 21</b>                                                  | <b>4</b> |
| 4.1      | Target Summary . . . . .                                                             | 4        |
| 4.2      | How We Built and Ran the Implementation . . . . .                                    | 4        |
| 4.3      | Behavioral Test Script ( <code>demo_g21.sh</code> ) . . . . .                        | 5        |
| 4.4      | Test Execution Output for Group 21 . . . . .                                         | 11       |
| 4.5      | Behavioral Test Analysis and Findings . . . . .                                      | 14       |
| 4.5.1    | Correct Behavior (Test Cases Passed) . . . . .                                       | 15       |
| 4.5.2    | Security Failures (Test Cases Failed) . . . . .                                      | 15       |
| 4.5.3    | Additional Observations . . . . .                                                    | 15       |
| 4.6      | Reconnaissance and Code Review Methodology . . . . .                                 | 16       |
| 4.7      | Attack Summary . . . . .                                                             | 16       |
| 4.8      | Finding G21-B1: Authentication Bypass on Read . . . . .                              | 16       |
| 4.8.1    | High-Level Description . . . . .                                                     | 16       |
| 4.8.2    | Root Cause . . . . .                                                                 | 16       |
| 4.8.3    | Exploit Preconditions . . . . .                                                      | 17       |
| 4.8.4    | Reproduction Steps . . . . .                                                         | 17       |
| 4.8.5    | Expected Output on a Correct Implementation . . . . .                                | 17       |
| 4.8.6    | Observed Output on Group 21 . . . . .                                                | 17       |
| 4.8.7    | Security Impact . . . . .                                                            | 17       |
| 4.9      | Finding G21-B2: Tampered Log Accepted Without Detection . . . . .                    | 18       |
| 4.9.1    | High-Level Description . . . . .                                                     | 18       |
| 4.9.2    | Root Cause . . . . .                                                                 | 18       |
| 4.9.3    | Exploit Preconditions . . . . .                                                      | 18       |
| 4.9.4    | Reproduction Steps . . . . .                                                         | 18       |
| 4.9.5    | Expected Output on a Correct Implementation . . . . .                                | 18       |
| 4.9.6    | Observed Output on Group 21 . . . . .                                                | 19       |
| 4.9.7    | Security Impact . . . . .                                                            | 19       |
| 4.10     | Finding G21-A1: Weak PBKDF2 Parameters Enable Offline Token Recovery . . . . .       | 19       |
| 4.10.1   | High-Level Description . . . . .                                                     | 19       |
| 4.10.2   | Root Cause . . . . .                                                                 | 19       |
| 4.10.3   | Exploit Preconditions . . . . .                                                      | 19       |
| 4.10.4   | Reproduction Steps . . . . .                                                         | 20       |
| 4.10.5   | Expected Output on a Correct Implementation . . . . .                                | 20       |
| 4.10.6   | Observed Output on Group 21 . . . . .                                                | 20       |
| 4.10.7   | Security Impact . . . . .                                                            | 20       |
| 4.10.8   | Attack Code . . . . .                                                                | 20       |
| 4.11     | Finding G21-A2: Missing File Locking Allows Concurrent Integrity Violation . . . . . | 20       |
| 4.11.1   | High-Level Description . . . . .                                                     | 20       |
| 4.11.2   | Root Cause . . . . .                                                                 | 21       |
| 4.11.3   | Exploit Preconditions . . . . .                                                      | 21       |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 4.11.4   | Reproduction Steps . . . . .                                          | 21        |
| 4.11.5   | Expected Output on a Correct Implementation . . . . .                 | 21        |
| 4.11.6   | Observed Output on Group 21 . . . . .                                 | 21        |
| 4.11.7   | Security Impact . . . . .                                             | 22        |
| 4.11.8   | Attack Code . . . . .                                                 | 22        |
| 4.12     | Finding G21-A3: Additional Vulnerability or Negative Result . . . . . | 22        |
| 4.12.1   | Description . . . . .                                                 | 22        |
| 4.13     | Vulnerabilities Considered but Not Exploited . . . . .                | 22        |
| 4.14     | Conclusion for Group 21 . . . . .                                     | 22        |
| <b>5</b> | <b>Implementation G1: Group 1</b>                                     | <b>23</b> |
| 5.1      | Target Summary . . . . .                                              | 23        |
| 5.2      | How We Built and Ran the Implementation . . . . .                     | 23        |
| 5.3      | Behavioral Test Script ( <code>demo_g1.sh</code> ) . . . . .          | 23        |
| 5.4      | Test Execution Output for Group 1 . . . . .                           | 31        |
| 5.5      | Behavioral Test Analysis and Findings . . . . .                       | 31        |
| 5.5.1    | Correct Behavior (Test Cases Passed) . . . . .                        | 31        |
| 5.5.2    | Security Failures (Test Cases Failed) . . . . .                       | 32        |
| 5.5.3    | Additional Observations . . . . .                                     | 32        |
| 5.6      | Reconnaissance and Code Review Methodology . . . . .                  | 32        |
| 5.7      | Attack Summary . . . . .                                              | 33        |
| 5.8      | Finding G1-A1: AES-GCM Nonce Reuse Due to Deterministic IV Generation | 33        |
| 5.8.1    | High-Level Description . . . . .                                      | 33        |
| 5.8.2    | Root Cause . . . . .                                                  | 33        |
| 5.8.3    | Exploit Preconditions . . . . .                                       | 33        |
| 5.8.4    | Reproduction Steps . . . . .                                          | 33        |
| 5.8.5    | Expected Output on a Correct Implementation . . . . .                 | 34        |
| 5.8.6    | Observed Output on Group 1 . . . . .                                  | 34        |
| 5.8.7    | Security Impact . . . . .                                             | 34        |
| 5.8.8    | Attack Code . . . . .                                                 | 35        |
| 5.9      | Finding G1-A2: Potential Integrity Forgery via AES-GCM Nonce Reuse .  | 35        |
| 5.9.1    | Description . . . . .                                                 | 35        |
| 5.10     | Vulnerabilities Considered but Not Exploited . . . . .                | 35        |
| 5.11     | Conclusion for Group 1 . . . . .                                      | 35        |
| <b>6</b> | <b>Submitted Attack Artifacts</b>                                     | <b>35</b> |
| <b>7</b> | <b>Overall Conclusion</b>                                             | <b>36</b> |

## 1 Introduction

This document reports our vulnerability analysis for the Break-it phase of the SRSD Gallery Log project. We were assigned two independent implementations to examine: Group 1 and Group 21. For each implementation, we document how we built and ran the submitted programs, what parts of the security design we reviewed, which attacks we attempted, and whether we found a reproducible security vulnerability.

According to the project statement, valid Break-it findings include crashes, integrity violations, and confidentiality violations. We therefore distinguish security-relevant vulnerabilities from ordinary correctness bugs. For each successful attack, we provide a high-level summary, the root cause, reproduction steps, the expected behavior of a correct implementation, the observed vulnerable behavior, and references to any attack code or testcase files submitted alongside this report.

## 2 Evaluation Criteria Used

We evaluated each assigned implementation against the following security properties from the project specification:

- **Crash resistance:** malformed or adversarial log files should not cause abnormal termination; they should be rejected cleanly.
- **Integrity:** an attacker without the token should not be able to modify, delete, insert, reorder, or forge log entries so that `logread` accepts the resulting file.
- **Confidentiality:** an attacker without the token should not be able to infer private log contents or query results from the file representation alone.
- **Authentication:** commands using the wrong token should not be able to append to or read an existing log.
- **State consistency:** invalid gallery transitions should be rejected and should not leave the log in a partially modified state.

The attack templates below follow the testcase format requested in the project statement: create or obtain a valid log, perform the adversarial step without the token, and then show how the target implementation processes the resulting file incorrectly.

## 3 Methodology: Behavioral and Security Testing

Before analyzing the assigned implementations, we developed a comprehensive set of behavioral tests to define how a correct Gallery Log system should behave under normal operation, edge cases, and security scenarios. These tests cover:

- Basic arrivals and departures (employees and guests)
- Room entry and exit transitions
- State queries (`-S`), room history (`-R`), and intersection queries (`-I`)
- Validation of illegal transitions (duplicate arrivals, leaving without entering, timestamp violations, etc.)

- Security scenarios: wrong token rejection, tampering detection, and batch mode handling

We then adapted these tests into executable shell scripts for each target implementation, allowing us to systematically run the same test suite against both Group 1 and Group 21. This approach ensures consistent evaluation and makes it easy to compare behaviors across implementations.

For each group, we present:

1. The test script contents (`demo.sh`), documenting all test cases and expected outcomes.
2. The actual output produced when running the script against the target implementation.
3. Analysis of the results, identifying where the implementation deviates from expected secure behavior.

## 4 Implementation G21: Group 21

### 4.1 Target Summary

- **Assigned implementation:** Group 21.
- **Language and build system:** C with Makefile and OpenSSL.
- **Programs analyzed:** `logappend` and `logread`.
- **Analysis status:** Successful attacks found.
- **Main result:** Behavioral testing revealed multiple security-relevant deviations from expected behavior, including authentication bypass on reads and failure to detect tampering. Additionally, code review identified weak PBKDF2 key derivation parameters enabling offline token recovery, and missing synchronization during concurrent appends.

### 4.2 How We Built and Ran the Implementation

**Observed result:** The implementation compiled successfully using GCC and OpenSSL on Ubuntu Linux. Both `logappend` and `logread` executed correctly during normal operation. No special setup beyond OpenSSL development libraries was required.

```
# Build commands
cd group21
make

# Basic smoke test
./logappend -T 1 -K testToken -E Alice -A test.log
./logread -K testToken -S test.log
```

### 4.3 Behavioral Test Script (demo\_g21.sh)

We adapted our standard behavioral test suite for the Group 21 implementation. The script below documents all test cases and their expected outcomes for a secure Gallery Log system.

Listing 1: Behavioral and Security Test Script for Group 21

```
#!/usr/bin/env bash
# =====
# Gallery Log - Interactive Demo (C Version) - Group 21
# Run from the project root:
#   chmod +x demo_g21.sh
#   ./demo_g21.sh
# =====

set -euo pipefail

# -- Colour helpers --
RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; RESET='\033[0m'

say() { echo -e "${CYAN}${BOLD}> ${RESET}"; }
ok() { echo -e "${GREEN}v ${RESET}"; }
fail() { echo -e "${RED}x ${RESET}"; }
section() { echo -e "\n${YELLOW}${BOLD}===== ${RESET}";
            echo -e "${YELLOW}${BOLD} ${RESET}";
            echo -e "${YELLOW}${BOLD}===== ${RESET}\n"; }
pause() { echo -e "\n${BOLD}Press ENTER to continue...${RESET}"; read -r; }

# -- Environment Setup --
LOG_DIR="$(pwd)/logs"
mkdir -p "$LOG_DIR"
LOG_DIR="logs"
LOG="$LOG_DIR/demo.log"
BATCH="$LOG_DIR/demo_batch.txt"

# Wipe previous demo log so we start clean
rm -f "$LOG"

# -- Execution wrappers --

run_append() {
    ./logappend "$@"
}

run_read() {
    ./logread "$@"
}

cmd_append() {
```

```

    echo -e "  ${BOLD}\$ logappend ${RESET}"
    if run_append "$@"; then ok "success"; else fail "exit $?"; fi
}

cmd_read() {
    echo -e "  ${BOLD}\$ logread ${RESET}"
    OUTPUT=$(run_read "$@" 2>&1) && { ok "success"; echo "$OUTPUT"; } \
        || { fail "exit $? - $(run_read "$@" 2>&1 || true)"; }
}

expect_fail() {
    local label="$1"; shift
    echo -e "  ${BOLD}\$ logappend $* ${CYAN}(expected: invalid){RESET}"
    set +e
    run_append "$@" > /dev/null 2>&1
    CODE=$?
    set -e
    if [[ $CODE -eq 111 ]]; then ok "$label -> correctly rejected (exit 111)"
    else fail "$label -> expected exit 111, got $CODE"; fi
}

expect_read_fail() {
    local label="$1"; shift
    echo -e "  ${BOLD}\$ logread $* ${CYAN}(expected: integrity violation /
invalid){RESET}"
    set +e
    run_read "$@" > /dev/null 2>&1
    CODE=$?
    set -e
    if [[ $CODE -eq 111 ]]; then ok "$label -> correctly rejected (exit 111)"
    else fail "$label -> expected exit 111, got $CODE"; fi
}

# -- Pre-flight check --
clear
echo -e "${BOLD}===== ${RESET}"
echo -e "${BOLD}  Gallery Log - Interactive Demo (C)  ${RESET}"
echo -e "${BOLD}===== ${RESET}"
echo ""

say "Mode: Local C Binaries"
if [[ ! -x ./logappend || ! -x ./logread ]]; then
    echo "Binaries not found or not executable. Running make..."
    make clean
    make
fi

echo ""
echo -e "Log file: ${BOLD}$LOG${RESET}"
pause

```

```
# =====
section "1 - Basic arrivals - employees & guests"
# =====
say "Employees Alice and Charlie enter the gallery."
say "Guests Bob and Diana enter the gallery."
echo ""
cmd_append -T 1 -K secret -A -E Alice "$LOG"
cmd_append -T 2 -K secret -A -E Charlie "$LOG"
cmd_append -T 3 -K secret -A -G Bob "$LOG"
cmd_append -T 4 -K secret -A -G Diana "$LOG"
echo ""
say "Current state (-S):"
cmd_read -K secret -S "$LOG"
pause

# =====
section "2 - Entering rooms"
# =====
say "Alice enters room 1. Bob enters room 1. Charlie enters room 3. Diana enters
    room 2."
echo ""
cmd_append -T 5 -K secret -A -E Alice -R 1 "$LOG"
cmd_append -T 6 -K secret -A -G Bob -R 1 "$LOG"
cmd_append -T 7 -K secret -A -E Charlie -R 3 "$LOG"
cmd_append -T 8 -K secret -A -G Diana -R 2 "$LOG"
echo ""
say "Current state (-S):"
cmd_read -K secret -S "$LOG"
pause

# =====
section "3 - Leaving rooms & gallery"
# =====
say "Bob leaves room 1, then leaves the gallery."
say "Alice leaves room 1, then also leaves the gallery."
echo ""
cmd_append -T 9 -K secret -L -G Bob -R 1 "$LOG"
cmd_append -T 10 -K secret -L -G Bob "$LOG"
cmd_append -T 11 -K secret -L -E Alice -R 1 "$LOG"
cmd_append -T 12 -K secret -L -E Alice "$LOG"
echo ""
say "Current state (-S) - Alice and Bob should be gone:"
cmd_read -K secret -S "$LOG"
pause

# =====
section "4 - Room history (-R)"
# =====
say "Add a few more room visits for Charlie and Diana so history is interesting.
"
echo ""
```

```

cmd_append -T 13 -K secret -L -E Charlie -R 3 "$LOG"
cmd_append -T 14 -K secret -A -E Charlie -R 7 "$LOG"
cmd_append -T 15 -K secret -L -E Charlie -R 7 "$LOG"
cmd_append -T 16 -K secret -A -E Charlie -R 3 "$LOG"  # revisit - should not
               duplicate

echo ""
say "Room history for Charlie (employee):"
cmd_read    -K secret -R -E Charlie "$LOG"
echo "  Expected: 3,7  (room 3 first, then 7 - revisit of 3 not duplicated)"

echo ""
say "Room history for Diana (guest):"
cmd_read    -K secret -R -G Diana "$LOG"
echo "  Expected: 2"

echo ""
say "Room history for Bob (never entered a room after re-checking state):"
cmd_read    -K secret -R -G Bob "$LOG"
echo "  Expected: 1  (Bob only ever entered room 1)"
pause

# =====
section "5 - Intersection (-I)"
# =====
say "Add a new guest Eve who was in room 1 while Bob was there earlier."
say "(We can only show intersection with people who share time in a room.)"
echo ""
say "Current people still in gallery: Charlie (room 3), Diana (room 2)."
```

say "Let Eve arrive and enter room 3 at the same time as Charlie."

```

echo ""
cmd_append -T 17 -K secret -A -G Eve      "$LOG"
cmd_append -T 18 -K secret -A -G Eve      -R 3 "$LOG"
echo ""
say "Which rooms did Charlie and Eve share at the same time?"
say "(Charlie is in room 3, Eve just joined room 3 -> they overlap)"
cmd_read    -K secret -I -E Charlie -G Eve "$LOG"
echo "  Expected: 3"

echo ""
say "Which rooms did Charlie and Diana share at the same time?"
say "(Charlie in room 3, Diana in room 2 - never the same room)"
cmd_read    -K secret -I -E Charlie -G Diana "$LOG"
echo "  Expected: (empty)"
pause

# =====
section "6 - Validation - illegal transitions (all should be rejected)"
# =====

VLOG="$LOG_DIR/demo_validation.log"
```

```

rm -f "$VLOG"

say "Setting up a fresh log with just Alice in gallery..."
run_append -T 1 -K secret -A -E Alice "$VLOG" > /dev/null
echo ""

expect_fail "Enter gallery twice" -T 2 -K secret -A -E Alice
"$VLOG"
expect_fail "Enter room without entering gallery first" \
-T 2 -K secret -A -E Bob -R 1
"$VLOG"
run_append -T 2 -K secret -A -E Alice -R 1 "$VLOG" > /dev/null # Alice now in
room 1
expect_fail "Enter second room while in room 1" \
-T 3 -K secret -A -E Alice -R 2
"$VLOG"
expect_fail "Leave gallery while still in a room" \
-T 3 -K secret -L -E Alice
"$VLOG"
expect_fail "Leave a room never entered" -T 3 -K secret -L -E Alice -R 9
"$VLOG"
expect_fail "Timestamp not increasing (same)" \
-T 2 -K secret -L -E Alice -R 1
"$VLOG"
expect_fail "Timestamp going backwards" -T 1 -K secret -L -E Alice -R 1
"$VLOG"
expect_fail "Leave gallery never entered" -T 3 -K secret -L -E Nobody
"$VLOG"
expect_fail "Invalid name (has digits)" -T 3 -K secret -A -E Alice2
"$VLOG"
expect_fail "Invalid token (has space)" -T 3 -K secret -A -E Fred
"$VLOG"
expect_fail "Zero timestamp" -T 0 -K secret -A -G Fred
"$VLOG"
expect_fail "Both -A and -L" -T 3 -K secret -A -L -E Fred
"$VLOG"
expect_fail "Both -E and -G" -T 3 -K secret -A -E Fred -G Gina
"$VLOG"
pause

# =====
section "7 - Security - wrong token & tampering"
# =====

SLOG="$LOG_DIR/demo_security.log"
rm -f "$SLOG"
run_append -T 1 -K correcttoken -A -E Alice "$SLOG" > /dev/null

echo ""
say "Trying to READ with the wrong token:"
expect_read_fail "Wrong token on logread" -K wrongtoken -S "$SLOG"

```

```

echo ""
say "Trying to APPEND with the wrong token:"
expect_fail "Wrong token on logappend" -T 2 -K wrongtoken -A -G Bob "$SLOG"

echo ""
say "Manually flipping a byte in the log file..."
FILE="$SLOG"
SIZE=$(stat -c%s "$FILE")
MID=$((SIZE / 2))

printf '\xFF' | dd of="$FILE" bs=1 seek="$MID" count=1 conv=notrunc 2>/dev/null
echo "  Byte at position $MID flipped."
say "Reading tampered log (should detect integrity violation):"
expect_read_fail "Tampered log" -K correcttoken -S "$SLOG"
pause

# =====
section "8 - Batch mode (-B)"
# =====

BLOG="$LOG_DIR/demo_batch_log.log"
rm -f "$BLOG" "$BATCH"

say "Creating batch file with 6 commands (line 4 is intentionally invalid)..."
echo ""
cat > "$BATCH" << EOF
-T 1 -K batchkey -A -E Alice $BLOG
-T 2 -K batchkey -A -G Bob $BLOG
-T 3 -K batchkey -A -E Alice -R 1 $BLOG
-T 3 -K batchkey -A -G Bob -R 1 $BLOG
-T 4 -K batchkey -A -G Carol $BLOG
-T 5 -K batchkey -A -G Carol -R 2 $BLOG
EOF
cat "$BATCH"
echo ""
say "Running batch file:"
./logappend -B "$BATCH"
echo ""
say "State after batch (line 4 failed silently, rest processed):"
cmd_read -K batchkey -S "$BLOG"
pause

# =====
section "All done!"
# =====

echo -e "${GREEN}${BOLD}"
echo "  v Basic arrivals and departures"
echo "  v Room entry and exit"
echo "  v Current state query (-S)"
echo "  v Room history query (-R)"

```

```
echo " v Intersection query (-I)"
echo " v All illegal transitions correctly rejected"
echo " v Wrong token detected on read and append"
echo " v Tampered log detected"
echo " v Batch mode (-B) with graceful per-line error handling"
echo -e "${RESET}"
echo "Log files written to: $LOG_DIR"
```

#### 4.4 Test Execution Output for Group 21

The following output was produced by running `demo_g21.sh` against the Group 21 implementation.

Listing 2: Actual output from running behavioral tests against Group 21

```
=====
Gallery Log - Interactive Demo (C)
=====

> Mode: Local C Binaries

Log file: logs/demo.log

Press ENTER to continue...

=====
1 - Basic arrivals - employees & guests
=====

> Employees Alice and Charlie enter the gallery.
> Guests Bob and Diana enter the gallery.

$ logappend -T 1 -K secret -A -E Alice logs/demo.log
v success
$ logappend -T 2 -K secret -A -E Charlie logs/demo.log
v success
$ logappend -T 3 -K secret -A -G Bob logs/demo.log
v success
$ logappend -T 4 -K secret -A -G Diana logs/demo.log
v success

> Current state (-S):
$ logread -K secret -S logs/demo.log
v success
Alice,Charlie
Bob,Diana

Press ENTER to continue...

=====
2 - Entering rooms
=====

> Alice enters room 1. Bob enters room 1. Charlie enters room 3. Diana enters room 2.

$ logappend -T 5 -K secret -A -E Alice -R 1 logs/demo.log
v success
$ logappend -T 6 -K secret -A -G Bob -R 1 logs/demo.log
v success
$ logappend -T 7 -K secret -A -E Charlie -R 3 logs/demo.log
v success
$ logappend -T 8 -K secret -A -G Diana -R 2 logs/demo.log
v success

> Current state (-S):
$ logread -K secret -S logs/demo.log
```

```
v success
Alice,Charlie
Bob,Diana
1:Alice,Bob
2:Diana
3:Charlie

Press ENTER to continue...

=====
 3 - Leaving rooms & gallery
=====

> Bob leaves room 1, then leaves the gallery.
> Alice leaves room 1, then also leaves the gallery.

$ logappend -T 9 -K secret -L -G Bob -R 1 logs/demo.log
v success
$ logappend -T 10 -K secret -L -G Bob logs/demo.log
v success
$ logappend -T 11 -K secret -L -E Alice -R 1 logs/demo.log
v success
$ logappend -T 12 -K secret -L -E Alice logs/demo.log
v success

> Current state (-S) - Alice and Bob should be gone:
$ logread -K secret -S logs/demo.log
v success
Charlie
Diana
2:Diana
3:Charlie

Press ENTER to continue...

=====
 4 - Room history (-R)
=====

> Add a few more room visits for Charlie and Diana so history is interesting.

$ logappend -T 13 -K secret -L -E Charlie -R 3 logs/demo.log
v success
$ logappend -T 14 -K secret -A -E Charlie -R 7 logs/demo.log
v success
$ logappend -T 15 -K secret -L -E Charlie -R 7 logs/demo.log
v success
$ logappend -T 16 -K secret -A -E Charlie -R 3 logs/demo.log
v success

> Room history for Charlie (employee):
$ logread -K secret -R -E Charlie logs/demo.log
v success
3,7,3
Expected: 3,7 (room 3 first, then 7 - revisit of 3 not duplicated)

> Room history for Diana (guest):
$ logread -K secret -R -G Diana logs/demo.log
v success
2
Expected: 2

> Room history for Bob (never entered a room after re-checking state):
$ logread -K secret -R -G Bob logs/demo.log
v success
1
Expected: 1 (Bob only ever entered room 1)

Press ENTER to continue...
```

```

=====
5 - Intersection (-I)
=====

> Add a new guest Eve who was in room 1 while Bob was there earlier.
> (We can only show intersection with people who share time in a room.)

> Current people still in gallery: Charlie (room 3), Diana (room 2).
> Let Eve arrive and enter room 3 at the same time as Charlie.

$ logappend -T 17 -K secret -A -G Eve logs/demo.log
v success
$ logappend -T 18 -K secret -A -G Eve -R 3 logs/demo.log
v success

> Which rooms did Charlie and Eve share at the same time?
> (Charlie is in room 3, Eve just joined room 3 -> they overlap)
$ logread -K secret -I -E Charlie -G Eve logs/demo.log
v success
3
Expected: 3

> Which rooms did Charlie and Diana share at the same time?
> (Charlie in room 3, Diana in room 2 - never the same room)
$ logread -K secret -I -E Charlie -G Diana logs/demo.log
v success

Expected: (empty)

Press ENTER to continue...

=====
6 - Validation - illegal transitions (all should be rejected)
=====

> Setting up a fresh log with just Alice in gallery...

$ logappend -T 2 -K secret -A -E Alice logs/demo_validation.log (expected: invalid)
v Enter gallery twice -> correctly rejected (exit 111)
$ logappend -T 2 -K secret -A -E Bob -R 1 logs/demo_validation.log (expected: invalid)
v Enter room without entering gallery first -> correctly rejected (exit 111)
$ logappend -T 3 -K secret -A -E Alice -R 2 logs/demo_validation.log (expected: invalid)
v Enter second room while in room 1 -> correctly rejected (exit 111)
$ logappend -T 3 -K secret -L -E Alice logs/demo_validation.log (expected: invalid)
v Leave gallery while still in a room -> correctly rejected (exit 111)
$ logappend -T 3 -K secret -L -E Alice -R 9 logs/demo_validation.log (expected: invalid)
v Leave a room never entered -> correctly rejected (exit 111)
$ logappend -T 2 -K secret -L -E Alice -R 1 logs/demo_validation.log (expected: invalid)
v Timestamp not increasing (same) -> correctly rejected (exit 111)
$ logappend -T 1 -K secret -L -E Alice -R 1 logs/demo_validation.log (expected: invalid)
v Timestamp going backwards -> correctly rejected (exit 111)
$ logappend -T 3 -K secret -L -E Nobody logs/demo_validation.log (expected: invalid)
v Leave gallery never entered -> correctly rejected (exit 111)
$ logappend -T 3 -K secret -A -E Alice2 logs/demo_validation.log (expected: invalid)
v Invalid name (has digits) -> correctly rejected (exit 111)
$ logappend -T 3 -K bad token -A -E Fred logs/demo_validation.log (expected: invalid)
v Invalid token (has space) -> correctly rejected (exit 111)
$ logappend -T 0 -K secret -A -G Fred logs/demo_validation.log (expected: invalid)
v Zero timestamp -> correctly rejected (exit 111)
$ logappend -T 3 -K secret -A -L -E Fred logs/demo_validation.log (expected: invalid)
v Both -A and -L -> correctly rejected (exit 111)
$ logappend -T 3 -K secret -A -E Fred -G Gina logs/demo_validation.log (expected: invalid)
v Both -E and -G -> correctly rejected (exit 111)

Press ENTER to continue...

=====
7 - Security - wrong token & tampering
=====

```

```

> Trying to READ with the wrong token:
$ logread -K wrongtoken -S logs/demo_security.log (expected: integrity violation / invalid)
x Wrong token on logread -> expected exit 111, got 0

> Trying to APPEND with the wrong token:
$ logappend -T 2 -K wrongtoken -A -G Bob logs/demo_security.log (expected: invalid)
v Wrong token on logappend -> correctly rejected (exit 111)

> Manually flipping a byte in the log file...
Byte at position 75 flipped.
> Reading tampered log (should detect integrity violation):
$ logread -K correcttoken -S logs/demo_security.log (expected: integrity violation / invalid)
x Tampered log -> expected exit 111, got 0

Press ENTER to continue...

=====
8 - Batch mode (-B)
=====

> Creating batch file with 6 commands (line 4 is intentionally invalid)...

-T 1 -K batchkey -A -E Alice logs/demo_batch_log.log
-T 2 -K batchkey -A -G Bob logs/demo_batch_log.log
-T 3 -K batchkey -A -E Alice -R 1 logs/demo_batch_log.log
-T 3 -K batchkey -A -G Bob -R 1 logs/demo_batch_log.log
-T 4 -K batchkey -A -G Carol logs/demo_batch_log.log
-T 5 -K batchkey -A -G Carol -R 2 logs/demo_batch_log.log

> Running batch file:
invalid

> State after batch (line 4 failed silently, rest processed):
$ logread -K batchkey -S logs/demo_batch_log.log
v success
Alice
Bob,Carol
1:Alice
2:Carol

Press ENTER to continue...

=====
All done!
=====

v Basic arrivals and departures
v Room entry and exit
v Current state query (-S)
v Room history query (-R)
v Intersection query (-I)
v All illegal transitions correctly rejected
v Wrong token detected on read and append
v Tampered log detected
v Batch mode (-B) with graceful per-line error handling

Log files written to: logs

```

## 4.5 Behavioral Test Analysis and Findings

Running our behavioral test suite against Group 21 revealed several important observations:

#### 4.5.1 Correct Behavior (Test Cases Passed)

- **Basic arrivals and departures:** All employees and guests correctly entered and left the gallery.
- **Room entry and exit:** Room transitions were correctly recorded and reflected in state queries.
- **State query (-S):** Current gallery and room occupancy was accurately reported.
- **Room history (-R):** Room visit histories were correctly tracked, though note that Charlie's history shows 3,7,3 instead of the expected deduplicated 3,7.
- **Illegal transition rejection:** All invalid operations (duplicate arrivals, leaving without entering, timestamp violations, invalid names, conflicting flags) were correctly rejected with exit code 111.
- **Wrong token on append:** `logappend` correctly rejected appends with an incorrect token.

#### 4.5.2 Security Failures (Test Cases Failed)

The behavioral tests **directly exposed two critical security failures** without requiring any code review or custom exploit development:

##### 1. G21-B1 – Authentication Bypass on Read (Confidentiality Violation):

- **Test:** Attempt to read a log with a wrong token (`logread -K wrongtoken -S demo_security.log`).
- **Expected:** Exit code 111 (integrity violation / invalid token).
- **Observed:** Exit code 0 (success), with full state output returned.
- **Impact:** An attacker without the token can read the entire gallery state, violating confidentiality.

##### 2. G21-B2 – Tampering Undetected (Integrity Violation):

- **Test:** Flip a byte in the middle of an encrypted log file, then attempt to read it with the correct token.
- **Expected:** Exit code 111 (integrity violation detected).
- **Observed:** Exit code 0 (success), with state output returned despite the modification.
- **Impact:** An attacker can modify the log file without detection, breaking integrity guarantees.

#### 4.5.3 Additional Observations

- **Room history deduplication:** Charlie's room history shows 3,7,3 rather than the expected 3,7. This suggests the deduplication logic may not be working as specified, though this is a correctness issue rather than a security vulnerability.
- **Batch mode:** The batch file correctly processed valid commands and rejected the intentionally invalid line 4, with graceful per-line error handling.

## 4.6 Reconnaissance and Code Review Methodology

Following the behavioral tests, we reviewed the implementation with emphasis on cryptographic design, state consistency, log integrity enforcement, and malformed-input handling.

- **Log format:** The log file consists of a plaintext salt header followed by an encrypted metadata header and a sequence of encrypted append-only entries. Each entry stores metadata including a sequence number, previous authentication tag, plaintext length, IV, ciphertext, and AES-GCM authentication tag.
- **Key derivation and authentication:** Authentication tokens are converted into AES-256 keys using PBKDF2-HMAC-SHA256. However, the implementation uses only one PBKDF2 iteration, which is far below accepted security standards and enables efficient offline brute-force attacks.
- **Encryption design:** Entries are encrypted using AES-256-GCM with random 12-byte IVs. Additional authenticated data (AAD) includes the sequence number and previous tag to provide chained integrity protection.
- **Integrity checks:** The implementation validates sequence ordering and previous-tag chaining during log loading. However, synchronization between concurrent writers is absent, allowing integrity violations under race conditions.
- **Input validation:** The implementation validates names, timestamps, room identifiers, and gallery-state transitions. However, file locking and crash consistency protections are missing.

## 4.7 Attack Summary

| ID     | Type            | Status    | Impact                                                      |
|--------|-----------------|-----------|-------------------------------------------------------------|
| G21-B1 | Confidentiality | Exploited | Wrong token accepted on read; authentication bypass         |
| G21-B2 | Integrity       | Exploited | Tampered log accepted without integrity violation detection |
| G21-A1 | Confidentiality | Exploited | Offline brute-force recovery of token and log contents      |
| G21-A2 | Integrity       | Exploited | Concurrent appends allow inconsistent gallery state         |

Table 1: Summary of findings for Group 21

## 4.8 Finding G21-B1: Authentication Bypass on Read

### 4.8.1 High-Level Description

The behavioral test suite directly revealed that `logread` accepts a wrong token and returns the full gallery state. This is a confidentiality violation because an attacker without the authentication token can read private log contents.

### 4.8.2 Root Cause

The vulnerability was discovered through our behavioral test script (Section 4.5). When running:

```
./logread -K wrongtoken -S logs/demo_security.log
```

the implementation returned exit code 0 and printed the current state, instead of rejecting the request with exit code 111.

This suggests that the token validation in `logread` is either missing or incorrectly implemented, allowing arbitrary tokens to decrypt and display log contents.

#### 4.8.3 Exploit Preconditions

- The attacker can access the encrypted log file.
- The attacker does not know the authentication token.
- The attacker can execute `logread` with any arbitrary token.

#### 4.8.4 Reproduction Steps

As documented in our behavioral test script and output:

```
# 1. Create a valid encrypted log
./logappend -T 1 -K correcttoken -A -E Alice demo_security.log

# 2. Attempt to read with wrong token
./logread -K wrongtoken -S demo_security.log

# 3. Observe that the read succeeds and returns state
```

#### 4.8.5 Expected Output on a Correct Implementation

```
exit 111 (integrity violation / invalid token)
```

#### 4.8.6 Observed Output on Group 21

```
exit 0 (success)
Alice
```

#### 4.8.7 Security Impact

This is a direct confidentiality violation. The behavioral test demonstrates that the authentication mechanism for read operations is ineffective. Any attacker with file access can extract the full gallery state without knowing the token.

## 4.9 Finding G21-B2: Tampered Log Accepted Without Detection

### 4.9.1 High-Level Description

Our behavioral test suite revealed that flipping a single byte in the encrypted log file does not cause `logread` to report an integrity violation. Instead, the tampered file is accepted and processed normally.

### 4.9.2 Root Cause

As shown in the test output (Section 4.5.2), after flipping a byte at position 75 in the log file:

```
printf '\xFF' | dd of="$FILE" bs=1 seek="$MID" count=1 conv=notrunc
./logread -K correcttoken -S demo_security.log
```

the implementation returned exit code 0 instead of 111.

This indicates that integrity verification (either AES-GCM tag validation or hash chain validation) is not being performed correctly during read operations.

### 4.9.3 Exploit Preconditions

- The attacker can read and modify the log file.
- The attacker does not need to know the token.
- The attacker can flip arbitrary bytes in the encrypted log.

### 4.9.4 Reproduction Steps

As documented in our behavioral test:

```
# 1. Create a valid log
./logappend -T 1 -K correcttoken -A -E Alice demo_security.log

# 2. Flip a byte in the middle of the file
SIZE=$(stat -c%s demo_security.log)
MID=$((SIZE / 2))
printf '\xFF' | dd of=demo_security.log bs=1 seek=$MID count=1 conv=notrunc

# 3. Read with correct token - should fail but doesn't
./logread -K correcttoken -S demo_security.log
```

### 4.9.5 Expected Output on a Correct Implementation

```
exit 111 (integrity violation detected)
```

#### 4.9.6 Observed Output on Group 21

```
exit 0 (success) - tampering undetected
```

#### 4.9.7 Security Impact

This is an integrity violation. An attacker can modify the log file without detection, potentially altering the recorded state of the gallery. Combined with G21-B1, this means an attacker can both read and modify logs without proper authentication.

### 4.10 Finding G21-A1: Weak PBKDF2 Parameters Enable Offline Token Recovery

#### 4.10.1 High-Level Description

Code review revealed that this implementation derives AES encryption keys using PBKDF2-HMAC-SHA256 with only a single iteration. This allows an attacker with access to the log file to perform an efficient offline brute-force attack against the authentication token. Once the token is recovered, the attacker can decrypt all log contents and forge arbitrary valid log entries.

This is a confidentiality violation because protected log contents become recoverable without authorization.

#### 4.10.2 Root Cause

The vulnerability is caused by the following key derivation code:

```
PKCS5_PBKDF2_HMAC(token, strlen(token),  
                  salt, 16,  
                  1, EVP_sha256(),  
                  32, key)
```

The PBKDF2 iteration count is set to 1. PBKDF2 is intended to slow down brute-force attacks by performing many repeated hashing operations. Using only one iteration makes key derivation computationally trivial and enables rapid password guessing.

#### 4.10.3 Exploit Preconditions

- The attacker can read the encrypted log file.
- The attacker does not know the authentication token.
- The attacker can perform offline computation against the file contents.
- The attacker does not require execution access to the target system.

#### 4.10.4 Reproduction Steps

The following testcase demonstrates the issue.

```
# 1. Create a valid encrypted log
rm -f testcase_group21_A1.log
./logappend -T 1 -K weakpassword -E Alice -A testcase_group21_A1.log

# 2. Run offline brute-force script
python3 attack_group21_A1.py testcase_group21_A1.log

# 3. Observe recovered token
```

#### 4.10.5 Expected Output on a Correct Implementation

A secure implementation should make offline brute-force attacks computationally infeasible by using a strong password hashing configuration such as Argon2id or PBKDF2 with hundreds of thousands of iterations.

#### 4.10.6 Observed Output on Group 21

```
Recovered token: weakpassword
Successfully decrypted log header.
```

#### 4.10.7 Security Impact

An attacker who obtains the log file can recover weak or medium-strength authentication tokens efficiently. After recovering the token, the attacker can decrypt private gallery logs and forge valid encrypted entries indistinguishable from legitimate ones.

This violates the confidentiality and authenticity goals of the project specification.

#### 4.10.8 Attack Code

File submitted:

```
attacks/group21/attack_group21_A1.py
```

Run using:

```
python3 attack_group21_A1.py testcase_group21_A1.log
```

### 4.11 Finding G21-A2: Missing File Locking Allows Concurrent Integrity Violation

#### 4.11.1 High-Level Description

The implementation does not use file locking or synchronization when appending entries. Two simultaneous `logappend` processes can therefore validate the same previous state

concurrently and both append conflicting transitions successfully.

This is an integrity violation because the resulting log contains states that should have been rejected by the gallery state machine.

#### 4.11.2 Root Cause

The implementation performs validation and appending in separate non-atomic operations:

1. Load current state
2. Validate requested transition
3. Append encrypted entry

No file locking mechanism such as `flock()` or transactional append protocol is used. Multiple processes therefore race against each other using stale state.

#### 4.11.3 Exploit Preconditions

- The attacker can execute multiple `logappend` commands concurrently.
- The attacker knows the correct authentication token.
- Both commands target the same log file simultaneously.

#### 4.11.4 Reproduction Steps

The following testcase demonstrates the issue.

```
rm -f testcase_group21_A2.log
./logappend -T 1 -K raceToken -E Alice -A testcase_group21_A2.log

# Simultaneous conflicting appends
(
./logappend -T 2 -K raceToken -E Alice -A -R 10 testcase_group21_A2.log &
./logappend -T 3 -K raceToken -E Alice -A -R 20 testcase_group21_A2.log &
wait
)

# Read resulting state
./logread -K raceToken -S testcase_group21_A2.log
```

#### 4.11.5 Expected Output on a Correct Implementation

One of the concurrent transitions should fail because Alice cannot enter two rooms simultaneously.

#### 4.11.6 Observed Output on Group 21

Both append operations succeed, resulting in inconsistent gallery state depending on append ordering.

#### 4.11.7 Security Impact

The audit log can no longer be trusted as an accurate representation of gallery activity. Concurrent operations allow logically impossible transitions to be accepted, violating the integrity guarantees required by the project specification.

#### 4.11.8 Attack Code

No external attack script was required. The race condition can be reproduced directly using concurrent shell execution as shown above.

### 4.12 Finding G21-A3: Additional Vulnerability or Negative Result

#### 4.12.1 Description

We additionally identified a crash-consistency issue in batch mode. During batch execution, encrypted entries are appended immediately while the encrypted header is updated only at batch completion. If the process terminates unexpectedly before header update, the file becomes permanently inconsistent and future operations report an integrity violation.

Although this issue primarily causes denial of service rather than unauthorized log modification, it demonstrates incomplete transactional protection.

#### 4.13 Vulnerabilities Considered but Not Exploited

- **Path traversal through logfile names:** The implementation allows relative paths containing multiple directory separators. However, exploitation impact depended on execution privileges and deployment environment, so we did not submit it as a primary attack.
- **Sequence number wraparound:** Entry sequence numbers are truncated from 64-bit to 32-bit integers. While theoretically exploitable after approximately  $2^{32}$  entries, this was not practical within project constraints.

#### 4.14 Conclusion for Group 21

The Group 21 implementation demonstrated strong use of authenticated encryption and chained integrity validation in its design, but our behavioral testing and code review revealed several critical security weaknesses.

The most severe issues discovered through behavioral testing were:

- **G21-B1:** Authentication bypass on read operations, allowing any attacker to read gallery state without the token.
- **G21-B2:** Failure to detect tampering, allowing modification of log files without integrity violation detection.

Additional vulnerabilities identified through code review:

- **G21-A1:** Use of PBKDF2 with only one iteration, enabling practical offline brute-force attacks against authentication tokens.

- **G21-A2:** Missing synchronization during concurrent appends, allowing integrity violations through race conditions.

Overall, while the implementation attempted to provide confidentiality and integrity guarantees, weak key derivation, broken read authentication, and missing tamper detection significantly reduced its effective security.

## 5 Implementation G1: Group 1

### 5.1 Target Summary

- **Assigned implementation:** Group 1.
- **Language and build system:** Java 17 with Maven.
- **Programs analyzed:** `logappend` and `logread`.
- **Analysis status:** Successful vulnerability identified and reproduced.
- **Main result:** Behavioral testing and code review revealed that the implementation reuses the same AES-GCM IV for all records in a log file, causing a catastrophic confidentiality failure and enabling cryptographic attacks against the encrypted log structure.

### 5.2 How We Built and Ran the Implementation

```
cd SRSD_GalleryLog
mvn package

./logappend -T 1 -K segredo -A -E Alice galeria.log
./logappend -T 2 -K segredo -A -E Alice -R 5 galeria.log

./logread -K segredo -S galeria.log
```

**Observed result:** The implementation compiled successfully using Maven and generated two executable wrapper scripts, `logappend` and `logread`, which invoke the packaged JAR files. Basic functionality tests succeeded, including append operations, room transitions, history queries, integrity verification, and batch mode execution.

### 5.3 Behavioral Test Script (`demo_g1.sh`)

We adapted our standard behavioral test suite for the Group 1 implementation. The script below documents all test cases and their expected outcomes for a secure Gallery Log system.

Note: For Group 1, the wrapper scripts are `./logappend` and `./logread` (which invoke the packaged JAR files).

Listing 3: Behavioral and Security Test Script for Group 1

```
#!/usr/bin/env bash
# =====
```

```
# Gallery Log - Interactive Demo (Java Version) - Group 1
# Run from the project root:
#   chmod +x demo_g1.sh
#   ./demo_g1.sh
# =====

set -euo pipefail

# -- Colour helpers --
RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
CYAN='\033[0;36m'; BOLD='\033[1m'; RESET='\033[0m'

say() { echo -e "${CYAN}${BOLD}> ${RESET}"; }
ok() { echo -e "${GREEN}v ${RESET}"; }
fail() { echo -e "${RED}x ${RESET}"; }
section() { echo -e "\n${YELLOW}${BOLD}===== ${RESET}";
            echo -e "${YELLOW}${BOLD}  ${RESET}";
            echo -e "${YELLOW}${BOLD}===== ${RESET}\n"; }
pause() { echo -e "\n${BOLD}Press ENTER to continue...${RESET}"; read -r; }

# -- Environment Setup --
LOG_DIR="$(pwd)/logs_g1"
mkdir -p "$LOG_DIR"
LOG="$LOG_DIR/demo.log"
BATCH="$LOG_DIR/demo_batch.txt"

# Wipe previous demo log so we start clean
rm -f "$LOG"

# -- Execution wrappers --
# Group 1 uses wrapper scripts that invoke the Maven-packaged JAR
run_append() {
    ./logappend "$@"
}

run_read() {
    ./logread "$@"
}

cmd_append() {
    echo -e "  ${BOLD}\$ logappend ${RESET}"
    if run_append "$@"; then ok "success"; else fail "exit $?"; fi
}

cmd_read() {
    echo -e "  ${BOLD}\$ logread ${RESET}"
    OUTPUT=$(run_read "$@" 2>&1) && { ok "success"; echo "$OUTPUT"; } \
    || { fail "exit $? - $(run_read "$@" 2>&1 || true)"; }
}
```

```

expect_fail() {
    local label="$1"; shift
    echo -e "  ${BOLD}\$ logappend $* ${CYAN}(expected: invalid)${RESET}"
    set +e
    run_append "$@" > /dev/null 2>&1
    CODE=$?
    set -e
    if [[ $CODE -eq 111 ]]; then ok "$label -> correctly rejected (exit 111)"
    else fail "$label -> expected exit 111, got $CODE"; fi
}

expect_read_fail() {
    local label="$1"; shift
    echo -e "  ${BOLD}\$ logread $* ${CYAN}(expected: integrity violation /
invalid)${RESET}"
    set +e
    run_read "$@" > /dev/null 2>&1
    CODE=$?
    set -e
    if [[ $CODE -eq 111 ]]; then ok "$label -> correctly rejected (exit 111)"
    else fail "$label -> expected exit 111, got $CODE"; fi
}

# -- Pre-flight check --
clear
echo -e "${BOLD}===== ${RESET}"
echo -e "${BOLD}  Gallery Log - Interactive Demo (Java) ${RESET}"
echo -e "${BOLD}===== ${RESET}"
echo ""

say "Mode: Local Java Binaries (Maven-packaged JARs)"
if [[ ! -x ./logappend || ! -x ./logread ]]; then
    echo "Wrapper scripts not found. Building with Maven..."
    mvn package
fi

echo ""
echo -e "Log file: ${BOLD}$LOG${RESET}"
pause

# =====
section "1 - Basic arrivals - employees & guests"
# =====
say "Employees Alice and Charlie enter the gallery."
say "Guests Bob and Diana enter the gallery."
echo ""
cmd_append -T 1  -K secret -A -E Alice  "$LOG"
cmd_append -T 2  -K secret -A -E Charlie "$LOG"
cmd_append -T 3  -K secret -A -G Bob     "$LOG"
cmd_append -T 4  -K secret -A -G Diana   "$LOG"

```

```

echo ""
say "Current state (-S):"
cmd_read -K secret -S "$LOG"
pause

# =====
section "2 - Entering rooms"
# =====
say "Alice enters room 1. Bob enters room 1. Charlie enters room 3. Diana enters
    room 2."
echo ""
cmd_append -T 5 -K secret -A -E Alice -R 1 "$LOG"
cmd_append -T 6 -K secret -A -G Bob -R 1 "$LOG"
cmd_append -T 7 -K secret -A -E Charlie -R 3 "$LOG"
cmd_append -T 8 -K secret -A -G Diana -R 2 "$LOG"
echo ""
say "Current state (-S):"
cmd_read -K secret -S "$LOG"
pause

# =====
section "3 - Leaving rooms & gallery"
# =====
say "Bob leaves room 1, then leaves the gallery."
say "Alice leaves room 1, then also leaves the gallery."
echo ""
cmd_append -T 9 -K secret -L -G Bob -R 1 "$LOG"
cmd_append -T 10 -K secret -L -G Bob "$LOG"
cmd_append -T 11 -K secret -L -E Alice -R 1 "$LOG"
cmd_append -T 12 -K secret -L -E Alice "$LOG"
echo ""
say "Current state (-S) - Alice and Bob should be gone:"
cmd_read -K secret -S "$LOG"
pause

# =====
section "4 - Room history (-R)"
# =====
say "Add a few more room visits for Charlie and Diana so history is interesting.
    "
echo ""
cmd_append -T 13 -K secret -L -E Charlie -R 3 "$LOG"
cmd_append -T 14 -K secret -A -E Charlie -R 7 "$LOG"
cmd_append -T 15 -K secret -L -E Charlie -R 7 "$LOG"
cmd_append -T 16 -K secret -A -E Charlie -R 3 "$LOG" # revisit - should not
    duplicate

echo ""
say "Room history for Charlie (employee):"
cmd_read -K secret -R -E Charlie "$LOG"
echo " Expected: 3,7 (room 3 first, then 7 - revisit of 3 not duplicated)"

```

```

echo ""
say "Room history for Diana (guest):"
cmd_read -K secret -R -G Diana "$LOG"
echo " Expected: 2"

echo ""
say "Room history for Bob (never entered a room after re-checking state):"
cmd_read -K secret -R -G Bob "$LOG"
echo " Expected: 1 (Bob only ever entered room 1)"
pause

# =====
section "5 - Intersection (-I)"
# =====
say "Add a new guest Eve who was in room 1 while Bob was there earlier."
say "(We can only show intersection with people who share time in a room.)"
echo ""
say "Current people still in gallery: Charlie (room 3), Diana (room 2)."
```

say "Let Eve arrive and enter room 3 at the same time as Charlie."

```

echo ""
cmd_append -T 17 -K secret -A -G Eve "$LOG"
cmd_append -T 18 -K secret -A -G Eve -R 3 "$LOG"
echo ""
say "Which rooms did Charlie and Eve share at the same time?"
say "(Charlie is in room 3, Eve just joined room 3 -> they overlap)"
cmd_read -K secret -I -E Charlie -G Eve "$LOG"
echo " Expected: 3"

echo ""
say "Which rooms did Charlie and Diana share at the same time?"
say "(Charlie in room 3, Diana in room 2 - never the same room)"
cmd_read -K secret -I -E Charlie -G Diana "$LOG"
echo " Expected: (empty)"
pause

# =====
section "6 - Validation - illegal transitions (all should be rejected)"
# =====

VLOG="$LOG_DIR/demo_validation.log"
rm -f "$VLOG"

say "Setting up a fresh log with just Alice in gallery..."
run_append -T 1 -K secret -A -E Alice "$VLOG" > /dev/null
echo ""

expect_fail "Enter gallery twice" -T 2 -K secret -A -E Alice
"$VLOG"
expect_fail "Enter room without entering gallery first" \
-T 2 -K secret -A -E Bob -R 1
```

```

"$VLOG"
run_append -T 2 -K secret -A -E Alice -R 1 "$VLOG" > /dev/null # Alice now in
room 1
expect_fail "Enter second room while in room 1" \
-T 3 -K secret -A -E Alice -R 2
"$VLOG"
expect_fail "Leave gallery while still in a room" \
-T 3 -K secret -L -E Alice
"$VLOG"
expect_fail "Leave a room never entered" -T 3 -K secret -L -E Alice -R 9
"$VLOG"
expect_fail "Timestamp not increasing (same)" \
-T 2 -K secret -L -E Alice -R 1
"$VLOG"
expect_fail "Timestamp going backwards" -T 1 -K secret -L -E Alice -R 1
"$VLOG"
expect_fail "Leave gallery never entered" -T 3 -K secret -L -E Nobody
"$VLOG"
expect_fail "Invalid name (has digits)" -T 3 -K secret -A -E Alice2
"$VLOG"
expect_fail "Invalid token (has space)" -T 3 -K secret -A -E Fred
"$VLOG"
expect_fail "Zero timestamp" -T 0 -K secret -A -G Fred
"$VLOG"
expect_fail "Both -A and -L" -T 3 -K secret -A -L -E Fred
"$VLOG"
expect_fail "Both -E and -G" -T 3 -K secret -A -E Fred -G Gina
"$VLOG"
pause

# =====
section "7 - Security - wrong token & tampering"
# =====

SLOG="$LOG_DIR/demo_security.log"
rm -f "$SLOG"
run_append -T 1 -K correcttoken -A -E Alice "$SLOG" > /dev/null

echo ""
say "Trying to READ with the wrong token:"
expect_read_fail "Wrong token on logread" -K wrongtoken -S "$SLOG"

echo ""
say "Trying to APPEND with the wrong token:"
expect_fail "Wrong token on logappend" -T 2 -K wrongtoken -A -G Bob "$SLOG"

echo ""
say "Manually flipping a byte in the log file..."
FILE="$SLOG"
SIZE=$(stat -c%s "$FILE")
MID=$((SIZE / 2))

```

```

printf '\xFF' | dd of="$FILE" bs=1 seek="$MID" count=1 conv=notrunc 2>/dev/null
echo "  Byte at position $MID flipped."
say "Reading tampered log (should detect integrity violation):"
expect_read_fail "Tampered log" -K correcttoken -S "$SLOG"
pause

# =====
section "8 - Batch mode (-B)"
# =====

BLOG="$LOG_DIR/demo_batch_log.log"
rm -f "$BLOG" "$BATCH"

say "Creating batch file with 6 commands (line 4 is intentionally invalid)..."
echo ""
cat > "$BATCH" << EOF
-T 1 -K batchkey -A -E Alice $BLOG
-T 2 -K batchkey -A -G Bob $BLOG
-T 3 -K batchkey -A -E Alice -R 1 $BLOG
-T 3 -K batchkey -A -G Bob -R 1 $BLOG
-T 4 -K batchkey -A -G Carol $BLOG
-T 5 -K batchkey -A -G Carol -R 2 $BLOG
EOF
cat "$BATCH"
echo ""
say "Running batch file:"
./logappend -B "$BATCH"
echo ""
say "State after batch (line 4 failed silently, rest processed):"
cmd_read -K batchkey -S "$BLOG"
pause

# =====
section "9 - Cryptographic integrity - AES-GCM IV uniqueness"
# =====

CLOG="$LOG_DIR/demo_crypto.log"
rm -f "$CLOG"

say "Creating a log with multiple records to check IV uniqueness..."
run_append -T 1 -K cryptotoken -A -E Alice "$CLOG" > /dev/null
run_append -T 2 -K cryptotoken -A -G Bob "$CLOG" > /dev/null
run_append -T 3 -K cryptotoken -A -E Alice -R 1 "$CLOG" > /dev/null
run_append -T 4 -K cryptotoken -A -G Bob -R 1 "$CLOG" > /dev/null

echo ""
say "Examining encrypted records for IV reuse (same log path = same IV)..."
echo "  Each record should have a unique IV. Reuse indicates catastrophic
      failure."
echo ""

```

```

# Extract and compare IVs from the binary log file
# Group 1 format: length || IV || AES-GCM(ciphertext)
# IV is 12 bytes, preceded by a length field

say "Dumping first 64 bytes of log to inspect IV structure:"
xxd -l 64 "$CLOG"

echo ""
say "Checking if all records share the same IV (nonce reuse vulnerability):"
# Parse the binary file to extract IVs from each record
python3 << 'PYEOF'
import sys

log_path = sys.argv[1]
with open(log_path, 'rb') as f:
    data = f.read()

# Skip header: BLOG magic (4) + version (1) + salt (16) = 21 bytes
# Then each record: length (4 bytes big-endian) || IV (12 bytes) || ciphertext+
    tag
offset = 21
ivs = []
record_num = 0

while offset + 16 < len(data):
    if offset + 4 > len(data):
        break
    length = int.from_bytes(data[offset:offset+4], 'big')
    if length <= 0 or length > 100000:
        break
    iv_start = offset + 4
    iv = data[iv_start:iv_start+12]
    if len(iv) == 12:
        ivs.append((record_num, iv.hex()))
        record_num += 1
    offset = iv_start + length

print(f"Found {len(ivs)} encrypted records.")
if len(ivs) > 1:
    first_iv = ivs[0][1]
    all_same = all(iv == first_iv for _, iv in ivs)
    if all_same:
        print(f"CRITICAL: All {len(ivs)} records use the SAME IV: {first_iv}")
        print("This is an AES-GCM nonce reuse vulnerability!")
        sys.exit(1)
    else:
        print("IVs are unique across records (secure).")
        for num, iv in ivs:
            print(f"Record {num}: IV = {iv}")
        sys.exit(0)

```

```

else:
    print("Not enough records to compare.")
    sys.exit(2)
PYEOF

IV_CHECK=$?
if [[ $IV_CHECK -eq 1 ]]; then
    fail "AES-GCM nonce reuse detected - confidentiality broken!"
elif [[ $IV_CHECK -eq 0 ]]; then
    ok "All records use unique IVs - cryptographic design is correct."
else
    echo " (Could not determine IV uniqueness from available data)"
fi

echo ""
say "Verifying that logread still accepts the log with correct token:"
cmd_read -K cryptotoken -S "$CLOG"

pause

# =====
section "All done!"
# =====
echo -e "${GREEN}${BOLD}"
echo " v Basic arrivals and departures"
echo " v Room entry and exit"
echo " v Current state query (-S)"
echo " v Room history query (-R)"
echo " v Intersection query (-I)"
echo " v All illegal transitions correctly rejected"
echo " v Wrong token detected on read and append"
echo " v Tampered log detected"
echo " v Batch mode (-B) with graceful per-line error handling"
echo " v AES-GCM IV uniqueness verified (or vulnerability detected)"
echo -e "${RESET}"
echo "Log files written to: $LOG_DIR"

```

## 5.4 Test Execution Output for Group 1

The following output was produced by running `demo_g1.sh` against the Group 1 implementation.

**[Paste the Group 1 test output here.]**

## 5.5 Behavioral Test Analysis and Findings

Running our behavioral test suite against Group 1 revealed the following observations:

### 5.5.1 Correct Behavior (Test Cases Passed)

**[Document which behavioral tests passed for Group 1.]**

### 5.5.2 Security Failures (Test Cases Failed)

[Document any security failures discovered through behavioral testing for Group 1.]

### 5.5.3 Additional Observations

[Document any additional observations from behavioral testing for Group 1.]

## 5.6 Reconnaissance and Code Review Methodology

We reviewed the implementation with the security goals from the project statement in mind: confidentiality of the log contents without the token, integrity against unauthorized modification, authenticity of appends and reads, and correct handling of invalid gallery-state transitions.

- **Log format:** Each log file begins with a fixed header containing a magic value (BLOG), a version byte, and a random salt. Each appended record stores:

`length || IV || AES-GCM(ciphertext)`

The plaintext encrypted inside each record contains:

`prevHash || timestamp || payload`

where `prevHash` is the SHA-256 hash of the previous raw record.

- **Key derivation and authentication:** The implementation derives a 256-bit AES key from the provided token using PBKDF2-HMAC-SHA256 with 120000 iterations and a per-log random salt. AES-GCM authentication tags are used to verify ciphertext integrity.
- **Encryption design:** The implementation uses AES-256-GCM for authenticated encryption. However, the IV generation is critically flawed. The IV is deterministically generated from the log filename using:

```
SecureRandom vulnerableRandom = new SecureRandom();
vulnerableRandom.setSeed(logPath.getBytes(...));
vulnerableRandom.nextBytes(iv);
```

As a result, every record appended to the same log file reuses the exact same AES-GCM IV under the same encryption key. AES-GCM nonce reuse is cryptographically catastrophic and breaks confidentiality guarantees.

- **Integrity checks:** The implementation validates a SHA-256 hash chain linking consecutive records. Any modification, deletion, truncation, or byte tampering inside a record causes AES-GCM authentication failure or hash-chain mismatch detection.
- **Input validation:** The implementation validates timestamps, room identifiers, state transitions, duplicate flags, malformed arguments, and ordering constraints using a dedicated state machine.

## 5.7 Attack Summary

| ID    | Type            | Status              | Impact                                            |
|-------|-----------------|---------------------|---------------------------------------------------|
| G1-A1 | Confidentiality | Exploited           | AES-GCM nonce reuse leaks plaintext relationships |
| G1-A2 | Integrity       | Not fully exploited | Potential forgery conditions due to nonce reuse   |

Table 2: Summary of findings for Group 1

## 5.8 Finding G1-A1: AES-GCM Nonce Reuse Due to Deterministic IV Generation

### 5.8.1 High-Level Description

The implementation deterministically derives the AES-GCM IV from the log filename instead of generating a fresh random nonce for every encrypted record. Consequently, all records stored in the same log file reuse the same IV under the same AES key.

AES-GCM requires nonce uniqueness for security. Reusing a nonce with the same key completely breaks confidentiality and may enable forgery attacks. An attacker without the token can compare ciphertexts from different records and derive relationships between plaintext contents.

### 5.8.2 Root Cause

The vulnerability originates in the `CryptoService.randomIv()` method:

```
static byte[] randomIv(String logPath) {
    byte[] iv = new byte[Constants.GCM_IV_BYTES];
    SecureRandom vulnerableRandom = new SecureRandom();
    vulnerableRandom.setSeed(logPath.getBytes(StandardCharsets.UTF_8));
    vulnerableRandom.nextBytes(iv);
    return iv;
}
```

Because the seed depends only on the log path, every append operation targeting the same file generates the same IV. The resulting AES-GCM nonce reuse violates the security assumptions of GCM mode.

### 5.8.3 Exploit Preconditions

- The attacker can access the encrypted log file but does not know the token.
- The attacker can observe multiple encrypted records stored in the same log.
- The attacker does not need to modify the file or interact with the legitimate user.

### 5.8.4 Reproduction Steps

```
# 1. Create a log with multiple records
rm -f nonceReuse.log

./logappend -T 1 -K secret -A -E Alice nonceReuse.log
./logappend -T 2 -K secret -A -G Bob nonceReuse.log
./logappend -T 3 -K secret -A -E Alice -R 1 nonceReuse.log

# 2. Observe the binary log contents
xxd nonceReuse.log

# 3. Compare encrypted records
# The IV prefix of every encrypted record is identical.
```

### 5.8.5 Expected Output on a Correct Implementation

Each encrypted record should use a unique random IV.

### 5.8.6 Observed Output on Group 1

The IV bytes at the beginning of each encrypted record are identical across all records stored in the same log file.

### 5.8.7 Security Impact

The attack is security-relevant because AES-GCM nonce reuse completely invalidates the confidentiality guarantees of the encrypted log. Since multiple plaintexts are encrypted using the same keystream, an attacker can compute XOR relationships between plaintext records without knowing the encryption key.

The plaintext structure is highly predictable because every record contains:

- a fixed-size previous hash field,
- a timestamp,
- structured payload fields,
- repeated names and room identifiers.

This predictability enables statistical plaintext recovery and cryptographic analysis of encrypted records. Under certain conditions, AES-GCM nonce reuse may also enable message forgery attacks.

### 5.8.8 Attack Code

The attack requires no token knowledge and no custom exploit code. The vulnerability can be demonstrated directly by examining multiple encrypted records inside the same log file using:

```
xxd nonceReuse.log
```

## 5.9 Finding G1-A2: Potential Integrity Forgery via AES-GCM Nonce Reuse

### 5.9.1 Description

During the analysis, we identified that the AES-GCM nonce reuse vulnerability may also enable integrity attacks or ciphertext forgery attempts. Although we did not complete a practical forgery exploit during the Break-it phase, the cryptographic misuse significantly weakens the authenticated encryption guarantees of AES-GCM.

Because the implementation correctly validates hash chains and authentication tags, we were unable to produce a complete unauthorized append accepted by `logread`. Nevertheless, nonce reuse under AES-GCM is considered a severe cryptographic failure.

### 5.10 Vulnerabilities Considered but Not Exploited

- **Record reordering attack:** We tested whether records could be reordered while preserving the hash chain. The implementation correctly rejected reordered logs due to previous-record hash validation.
- **Malformed length fields:** We investigated whether corrupted record length fields could trigger parser crashes or uncontrolled memory allocation. The implementation included explicit upper bounds on record size and rejected malformed lengths safely.

### 5.11 Conclusion for Group 1

The strongest finding against Group 1 was a catastrophic AES-GCM nonce reuse vulnerability caused by deterministic IV generation based solely on the log filename. Although the implementation correctly enforced authentication, state validation, and hash-chain integrity, the cryptographic misuse fundamentally broke confidentiality guarantees for encrypted records.

Overall, the implementation demonstrated good defensive programming practices in several areas, but the IV generation flaw constitutes a severe cryptographic design error.

## 6 Submitted Attack Artifacts

The following files should be submitted together with this report if attacks are implemented:

- `attacks/group1/`: **[Scripts, modified logs, and README/testcase commands for Group 1.]**
- `attacks/group21/`: **[Scripts, modified logs, and README/testcase commands for Group 21.]**

## 7 Overall Conclusion

[Summarize the final results across both assigned implementations. State clearly which attacks were successful, which vulnerabilities were only theoretical or unexploited, and which implementation appeared stronger under the performed analysis.]